

Python Theory Interview Questions (0-3 Years Experience)

A curated list of theory questions commonly asked for Python developer roles targeting freshers and candidates with up to 3 years of experience.

1. Python Basics & Data Types

1. What is Python? What are its key features?
 2. Is Python compiled or interpreted? Explain.
 3. What are the built-in data types in Python?
 4. What is the difference between a list, a tuple, and a set?
 5. What is the difference between a list and a dictionary?
 6. Are Python strings mutable or immutable? Explain with an example.
 7. What is the difference between `is` and `==`?
 8. What is type conversion? Explain implicit vs explicit type conversion.
 9. What does the `id()` function do?
 0. What is the difference between `range()` and `xrange()`? (Python 2 vs 3)
 1. How is memory managed in Python?
 2. What is the difference between mutable and immutable objects? Give examples.
-

2. Operators & Control Flow

1. What are the different types of operators in Python?
 2. What is the difference between `/`, `//`, and `%`?
 3. What does the `**` operator do?
 4. Explain the use of the ternary (conditional) operator in Python.
 5. What is the difference between `break`, `continue`, and `pass`?
 6. Does Python have a `switch` statement? How do you achieve similar behavior?
 7. What is the purpose of the `else` block in loops?
-

3. Strings

1. How do you reverse a string in Python?
 2. What are string slicing and indexing?
 3. What is the difference between `find()` and `index()`?
 4. What are f-strings? How do they differ from `format()` and `%` formatting?
 5. What does the `join()` method do?
 6. How do you check if a string is a palindrome?
 7. What is the difference between `strip()`, `lstrip()`, and `rstrip()`?
-

4. Functions

1. What is a function? How do you define one in Python?
 2. What is the difference between arguments and parameters?
 3. What are `*args` and `**kwargs`?
 4. What are default arguments? What is the "mutable default argument" pitfall?
 5. What is a lambda function? When would you use one?
 6. What is the difference between `return` and `print`?
 7. What is variable scope? Explain local, global, and the `global` / `nonlocal` keywords.
 8. What are `map()`, `filter()`, and `reduce()`?
 9. What is recursion? Give an example.
 0. What is a docstring?
-

5. Object-Oriented Programming (OOP)

1. What are the four pillars of OOP? Explain each.
2. What is the difference between a class and an object?

3. What is `self` in Python?
 4. What is `__init__`? Is it a constructor?
 5. What is inheritance? What are its types in Python?
 6. What is method overriding vs method overloading? Does Python support overloading?
 7. What is the difference between an instance variable and a class variable?
 8. What are `@staticmethod` and `@classmethod`? How do they differ?
 9. What are dunder (magic) methods? Give examples like `__str__` and `__repr__`.
 10. What is encapsulation? How do you achieve private members in Python?
 1. What is polymorphism? Give an example.
 2. What is the difference between `super()` and directly calling the parent class?
 3. What is the MRO (Method Resolution Order)?
-

6. Exception Handling

1. What is exception handling? Explain `try`, `except`, `else`, and `finally`.
 2. What is the difference between an error and an exception?
 3. How do you raise a custom exception?
 4. What is the difference between `Exception` and `BaseException`?
 5. Can you have multiple `except` blocks? How?
 6. What does the `finally` block guarantee?
-

7. Modules, Packages & File Handling

1. What is the difference between a module and a package?
 2. What does `import` do? Difference between `import x` and `from x import y`?
 3. What is `__name__ == "__main__"` used for?
 4. What is `pip`? What is a virtual environment and why use it?
 5. How do you read from and write to a file in Python?
 6. What are the different file modes (`r`, `w`, `a`, `r+`, etc.)?
 7. What is the `with` statement (context manager) and why is it preferred for files?
-

8. Advanced Concepts (Commonly Asked at 1-3 Years)

1. What is a list comprehension? Give an example.
 2. What are dictionary and set comprehensions?
 3. What is a generator? How does it differ from a normal function?
 4. What is the difference between `yield` and `return`?
 5. What is an iterator? What is the difference between iterable and iterator?
 6. What is a decorator? Write a simple example.
 7. What is a closure?
 8. What is the Global Interpreter Lock (GIL)? How does it affect multithreading?
 9. What is the difference between multithreading and multiprocessing in Python?
 10. What is the difference between deep copy and shallow copy?
 1. What is monkey patching?
 2. What are Python's memory management and garbage collection mechanisms?
 3. What is the difference between `@property` and a regular method?
 4. What are `*` and `**` used for in function calls (unpacking)?
-

9. Standard Library & Practical

1. What is the difference between `append()` and `extend()`?
 2. How do `sort()` and `sorted()` differ?
 3. What does the `enumerate()` function do?
 4. What does the `zip()` function do?
 5. What is the difference between `remove()`, `pop()`, and `del`?
 6. How do you handle JSON in Python?
 7. What are Python's common built-in functions you use daily?
 8. What is slicing with a step, e.g. `a[::-1]` or `a[::2]`?
-

10. Quick Coding-Theory Questions

1. How do you remove duplicates from a list?
 2. How do you merge two dictionaries?
 3. How do you swap two variables without a temp variable?
 4. How do you count the frequency of elements in a list?
 5. What is the output of `print(2 == 2.0)` and why?
 6. What does `bool([])`, `bool(0)`, `bool("")` return?
 7. Explain the output of `[] == []` vs `[] is []`.
-

Tips for Candidates

- Be ready to give short code examples for most theory questions.
- Know the "why" behind concepts (e.g., why tuples are immutable, why GIL exists).
- Practice explaining OOP and generators/decorators clearly — these are the

most common differentiators between fresher and experienced candidates.